

DESARROLLO WEB EN ENTORNO CLIENTE

4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y
OBJETOS DEFINIDOS POR EL USUARIO

4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

1. FUNCIONES PREDEFINIDAS DEL LENGUAJE

`eval(cadena [,objeto ])`

- **Cadena:** Una cadena que representa una expresión, sentencia o secuencia de sentencias en JavaScript. La expresión puede incluir variables y propiedades de objetos existentes.
- **Objeto:** Un argumento opcional; si se especifica, la evaluación se restringe al contexto del objeto especificado.

Ejemplos:

1. `eval(new String("2 + 2"));` // devuelve un objeto String que contiene "2 + 2"
2. `eval("2 + 2");` // devuelve 4
3. `var expresion = new String("2 + 2");`
`eval(expresion.toString());`



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

1. FUNCIONES PREDEFINIDAS DEL LENGUAJE

`parseInt(cadena, base);`

- **Cadena:** Una cadena que representa el valor que se desea convertir.
- **Base:** Un entero que representa la base de la mencionada cadena.

Si se encuentra con un carácter que no se corresponde con la base especificada devuelve NaN.

Si no se especifica la base o se especifica como 0, JavaScript asume lo siguiente:

- Si el parámetro cadena comienza por "0x", la base es 16 (hexadecimal).
- Si el parámetro cadena comienza por "0", la base es de 8 (octal). Esta característica está desaconsejada.
- Si el parámetro cadena comienza por cualquier otro valor, la base es 10 (decimal).



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.1. FUNCIONES PREDEFINIDAS DEL LENGUAJE

Ejemplos:

1. `parseInt("F", 16);` //Devuelve 15
2. `parseInt("17", 8);` //Devuelve 15
3. `parseInt("15", 10);` //Devuelve 15
4. `parseInt(15.99, 10);` //Devuelve 15
5. `parseInt("Hello", 8);` // No es un número en absoluto.
Devuelve NaN
6. `parseInt("0x7", 10);` // Devuelve 7? Devuelve NaN?
No, devuelve 0, resultado del producto entre 0 y 7.



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

1. FUNCIONES PREDEFINIDAS DEL LENGUAJE

`parseFloat(cadena);`

- Cadena: Una cadena que representa el valor que se desea convertir.

Si el primer carácter no se puede convertir a número, devuelve NaN.

Ejemplos:

1. `parseFloat("3.14"); //Devuelve 3,14`
2. `parseFloat("314e-2"); //Devuelve 3,14`
3. `parseFloat("0.0314E+2"); //Devuelve 3,14`
4. `v var cadena = "3.14"; parseFloat(cadena); //Devuelve 3,14`
5. `parseFloat("3.14más caracteres no dígitos"); //Devuelve 3,14`
6. `parseFloat("FF2"); //Devuelve NaN`

4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

1. FUNCIONES PREDEFINIDAS DEL LENGUAJE

`isNaN(valor)`

- Valor: El valor que se desea evaluar.

isNaN quiere decir: Is Not a Number.

Intenta convertir el argumento a número; si no puede, devuelve true; en caso contrario false.

Ejemplos:

1. `isNaN(NaN) //devuelve true`
2. `isNaN("string") //devuelve true`
3. `isNaN("12") //devuelve false`
4. `isNaN(12) //devuelve false`



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

1. FUNCIONES PREDEFINIDAS DEL LENGUAJE

`isFinite(numero)`

- Valor: El valor que se desea evaluar.

Si el argumento es NaN, infinito positivo o negativo devuelve false; en caso contrario true.

Ejemplos:

1. `isFinite(Infinity); // falso`
2. `isFinite(NaN); // falso`
3. `isFinite(-Infinity); // falso`
4. `falso isFinite(0); // verdadero`
5. `isFinite(2e64); // verdadero`
6. `isFinite("0"); // verdadero`, hubiera sido falso en el caso de usar `Number.isFinite("0")` que es más robusta.



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

1. FUNCIONES PREDEFINIDAS DEL LENGUAJE

Number(objeto) / String(objeto)

- Objeto: el objeto que quiere pasarse a Number o String.

Convierten el objeto pasado por parámetro a Number o String.

Ejemplo:

```
var hoy = new Date();  
document.write(String(hoy));
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.2. FUNCIONES DEFINIDAS POR EL USUARIO

Son un conjunto de sentencias que realizan una tarea específica.

```
function nombre([argumentos]) {  
    //Sentencias  
    //[return dato]  
}
```

- **Function:** palabra reservada para definir una función.
- **Nombre:** palabra que determina el nombre de la función.
- **Argumentos (opcional):** parámetros que pueden utilizarse internamente en la función.
- **Return dato:** la función puede o no devolver un valor, que estaría indicado después de la palabra reservada return.



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.2. FUNCIONES DEFINIDAS POR EL USUARIO

Ejemplos:

```
1. function mifuncion(cadena) {  
    document.write("<P><BLINK>" + cadena + "</BLINK>");  
}
```

```
2. function resta(a,b) {  
    return a-b;  
}
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.2. FUNCIONES DEFINIDAS POR EL USUARIO

Arrow functions o funciones flecha

Alternativa compacta a una función convencional.

No son adecuadas para ser usadas como métodos y no pueden ser usadas como constructores.

SINTAXIS:

- `([param1, param2,...paramN]) => { sentencias }`
- `([param1, param2,...paramN]) => { return expresion }`
- `([param1, param2,...paramN]) => expresion`
- Paréntesis opcionales con un solo parámetro
`parametro => { sentencias }`
- Paréntesis obligatorios si la función no tiene parámetros
`() => { sentencias }`



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.2. FUNCIONES DEFINIDAS POR EL USUARIO

Arrow functions o funciones flecha - Ejemplos

```
var miFuncion = function () {  
    return new Date();  
}
```

```
var miFuncion = () => new Date();
```

```
var arraysConcatenados = function (array1, array2)  
{  
    return array1.concat(array2);  
}
```

```
var arraysConcatenados = (array1, array2) =>  
array1.concat(array2);
```

```
console.log(arraysConcatenados([1,2] , [3,4,5]));  
//Muestra por consola Array(5) [1, 2, 3,  
4, 5]
```

```
console.log(arraysConcatenados([1,2] , [3,4,5]));  
//Muestra por consola Array(5) [1, 2, 3,  
4, 5]
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.2. FUNCIONES DEFINIDAS POR EL USUARIO

Parámetros con valores por defecto

Se puede asignar un valor por defecto para un parámetro para los casos en que no se ingrese dicho argumento.

SINTAXIS:

```
function miFuncion (parámetro = valor) { sentencias }
```

EJEMPLO:

```
function raiz (número, base=2) { sentencias }
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.2. FUNCIONES DEFINIDAS POR EL USUARIO

Parámetros REST

Se puede definir parámetros que reciban un número indefinido de valores. Se creará un array que almacene todos estos valores.

SINTAXIS:

```
function miFuncion (...parámetro) { sentencias }
```

EJEMPLO:

```
function sumatorio (...sumandos) {  
    let total = 0;  
    for (const arg of sumandos) { total +=  
arg;    }  
    return total;  
}
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Arrays:

Colección de valores homogénea o no, y ordenada.

Crear un array usando el constructor del objeto Array:

```
miArray = new Array("Hola",30, 3.14); //Array de elementos  
heterogéneos miArray2 = new Array("Viento", "Lluvia", "Fuego",  
"Tierra"); //Array de elementos homogéneos  
miArray3 = new Array(5); //Array vacío con 5 elementos
```

Crear un array usando corchetes:

```
var frutas = ["Manzana", "Platano"]; console.log(frutas.length);  
// 2
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Arrays:

Colección de valores homogénea o no, y ordenada.

Acceder a un elemento de un array: `var primero = frutas[0];`

`// Manzana`

`var ultimo = frutas[frutas.length - 1]; // Platano`

Recorrer un array:

`frutas.forEach(function (item, index, array) { console.log(item,
index);`

`});`

`// Manzana 0`

`// Platano 1`



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Propiedades

Array

Propiedad	Descripción
length	Devuelve la longitud del array
prototype	Permite agregar al objeto Array las propiedades que queramos.

Ejemplo de prototype:

```
Array.prototype.descriptor = null;  
dias = new Array ('lunes', 'Martes', 'Miercoles', 'Jueves', 'Viernes'); dias.descriptor =  
"Dias laborables de la semana";
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Métodos

Array

Método	Descripción
valueOf(elemento)	Es el método por defecto de un objeto array y lo devuelve.
indexOf(elemento)	Devuelve la primera posición en la que se encuentra un elemento.
push(elemento)	Añade un elemento al final del array (aumenta su longitud).
pop()	Elimina un elemento al final del array y lo devuelve (disminuye su longitud)
unshift(elemento)	Añade un elemento al principio del array (aumenta su longitud)
shift()	Elimina el primer elemento del array y lo devuelve (disminuye su longitud).
reverse()	Modifica el array colocando los elementos en orden inverso.
sort()	Ordena los elementos del array según orden lexicográfico.

4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Métodos

Array

MétodoDescripción	
concat(elemento/s)	Concatena elementos de varios arrays.
join(separador)	Une todos los elementos en una cadena de texto (lo contrario que el split) unidos por un separador.
slice(inicio,fin)	Toma elementos de un array (sin modificar el original) desde la posición inicio a fin.
splice(inicio,numer o_elementos,[reem plazo])	Elimina un número de elementos de un array empezando por la posición inicio. Si tiene el tercer parámetro, además, los sustituye por él.

4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Partimos del array frutas["Manzana", "Platano"];

Obtener la posición de un elemento del array.

```
var posicion= frutas.indexOf("Platano");  
    // Devuelve 1, porque plátano es la 2ª posición, pero empezamos  
    por el 0.
```

Añadir al final del array:

```
var nuevoTamano = frutas.push("Naranja");  
    // ["Manzana", "Platano", "Naranja"] y devuelve el tamaño del  
    array
```

Eliminar del final del array:

```
var ultimo = frutas.pop(); // borra Naranja (del final) y lo devuelve  
    // ["Manzana", "Platano"];
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Añadir al principio del array:

```
var nuevoTamano = frutas.unshift("Fresa") // añade al principio y  
devuelve el tamaño  
// ["Fresa", "Manzana", "Platano"];
```

Eliminar del principio del array:

```
var primero = frutas.shift(); // borra Fresa(del principio) y lo devuelve  
// ["Manzana", "Platano"];
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Ordena según el orden lexicográfico:

```
var frutas=["Manzana","Platano","Kiwi","Higo","Pera","Ciruela"];  
var frutasOrdenado = frutas.sort ();  
    //frutas=["Ciruela", "Higo", "Kiwi", "Manzana", "Pera",  
    "Platano"]  
    //Devuelve el array ordenado
```

Invierte el orden de los elementos del array:

```
var frutas=["Manzana","Platano","Kiwi","Higo","Pera","Ciruela"];  
var frutasReves = frutas.reverse();  
    //frutas=["Ciruela", "Pera", "Higo", "Kiwi", "Platano",  
    "Manzana"]  
    //Devuelve el array ordenado del revés
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Concatenar dos arrays:

```
var frutas1 = ["Manzana", "Platano"];  
var frutas2 = ["Ciruela"];  
frutas1.concat("Kiwi", "Higo", "Pera");  
    //frutas1=["Manzana", "Platano", "Kiwi", "Higo", "Pera"];  
frutas1.concat(frutas2);  
    //frutas1=["Manzana", "Platano", "Kiwi", "Higo", "Pera", "Cir  
    uela"];
```

Unir los elementos de un array en una cadena:

```
var frutas=["Manzana", "Platano", "Kiwi"];  
var cadena = frutas.join(":");  
    // cadena = "Manzana:Platano:Kiwi";
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Tomar elementos según su posición (inicial y final):

```
var frutas=["Manzana","Platano","Kiwi", "Pera","Higo"];  
var seleccion = frutas.slice(0, 2); // seleccion =  
["Manzana","Platano"]  
var ultimos = frutas.slice(-2); // seleccion = ["Pera","Higo"]
```

Eliminar elementos según su posición (inicio, número de elementos):

```
var frutas=["Manzana","Platano","Kiwi", "Pera","Higo"];  
var eliminado= frutas.splice(2, 2); // frutas =  
["Manzana","Platano","Higo"]  
var ultimos = frutas.splice(-1); // frutas = ["Manzana", "Platano"]
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Arrays multidimensionales:

Son arrays que almacenan en cada posición otros arrays a su vez. Es posible crear arrays bidimensionales, tridimensionales, etc.

Crear un array bidimensional mediante otros arrays:

```
var datos = new Array();  
datos [0] = new Array ("Programacion", "1DAW", 10);  
datos[1] = new Array ("DWECE", "2DAW", 9);  
datos[2] = new Array ("DIW", "2DAW", 5);
```

Crear un array bidimensional usando corchetes:

```
var datos = [  
    ["Programacion", "1DAW", 10],  
    ["DWECE", "2DAW", 9],  
    ["DIW", "2DAW", 5]  
];
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Acceder a una posición de un array multidimensional:

Tenemos que indicar tantas posiciones como dimensiones (índices) tenga el array:

```
var elemento = datos [0][2];
```

Imprimir datos de un array multidimensional:

Utilizamos tantos bucles for anidados como dimensiones (índices) tenga el array.

```
for (i=0; i<datos.length; i++){  
    for (j=0; j<datos[i].length; j++) {  
        document.write("Elemento "+i+", "+j+":  
            "+datos[i][j]);  
    }  
}
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.3. ARRAYS

Operador Spread:

Permite a elementos iterables (arrays, strings...) u objetos ser expandidos para tomar por separado sus valores.

Esto puede ser útil cuando se esperan 0 o más argumentos como por ejemplo en llamadas a funciones o para realizar copias literales de arrays u objetos.

SINTAXIS: Al igual que en los parámetros REST el operador se especifica mediante "...".

Ejemplos:

```
let array1 = [1, 5, 7];  
let array2 = [2, 6, 8];  
console.log(Math.max(...array1, 3, ...array2, 4));    //Devuelve 8
```

```
let array3 = [...array1, ...array2];                  //Concatenar dos arrays en uno
```

```
let arrayCopia = [...array3];                          //Copia independiente de un  
array sin usar .slice()
```

```
let coche = {marca: "BMW", modelo: "M2"};  
let objetoCopia = {...coche};                          //Copia literal de un objeto
```

4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.4. OBJETOS

Objetos:

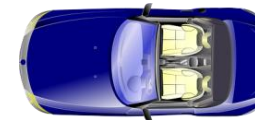
En Javascript, un **objeto** es una colección de propiedades que a su vez pueden ser datos o métodos.

Constructor: es una función especial para crear un objeto. Empieza en mayúscula:

```
class Coche {  
    //Propiedades y métodos  
}
```

Creación de un objeto:

```
var unCoche = new Coche([propiedades]);
```



```
let cocheAzul = new Coche();
```



```
let cocheRojo = new Coche();
```



```
let cocheVerde = new Coche();
```

4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.4. OBJETOS

Definir propiedades del objeto:

Se debe definir primero el constructor y como parámetros de éste las propiedades.

“Constructor” sin parámetros:

```
class Coche {  
  constructor() {  
    this.marca = ""; //Vacío  
    this.modelo = ""; //Vacío  
    this.combustible = "Diesel"; //Por defecto  
    this.kilometraje = 0; // Por defecto  
  }  
}
```



Crear un objeto vacío (sin propiedades):

```
let cocheVacio = new Coche();
```

Cambiar valores en las propiedades:

```
cocheVacio.marca = "Seat";  
cocheVacio.modelo = "Ibiza";  
cocheVacio.combustible = "Diesel";  
cocheVacio.kilometraje = 40;  
//Para ello debemos haber programado el método set en la clase
```

“Constructor” con parámetros:

```
class Coche {  
  constructor(marca, modelo, combustible, kilometraje) {  
    this.marca = marca;  
    this.modelo = modelo;  
    this.combustible = combustible;  
    this.kilometraje = kilometraje;  
  }  
  //Cada propiedad toma los valores recibidos por parámetro  
}
```



Crear un objeto inicializado (con propiedades):

```
let miCoche = new Coche("Seat", "Ibiza", "Diesel", 40000);
```

Acceder a las propiedades:

```
document.write(" Mi coche es un "+miCoche.marca+"  
"+miCoche.modelo);  
//Para ello debemos haber programado el método get en la clase
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.4. OBJETOS

Getters y setters

Permiten acceder y modificar las propiedades de los objetos.

Se utilizan mediante las palabras reservadas `get` y `set` seguidas del nombre de la función y paréntesis [y parámetros].

El nombre de los `getters` y `setters` no puede ser el mismo que la propiedad porque se produciría un bucle, por ello muchos desarrolladores utilizan el guión bajo para nombrar la propiedad.

```
class Coche
{
    constructor(marca) {
        this._marca
    }

    get marca() {
        return this._marca;
    }

    set marca(mar) {
        this._marca = mar;
    }
}
```

```
let miCoche = new Coche("Renault");

console.log(miCoche.marca);
//Muestra por consola Renault

miCoche.marca = "Toyota";
//Cambia el valor de la propiedad _marca a
Toyota

console.log(miCoche.marca);
//Muestra por consola Toyota
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.4. OBJETOS

Herencia

Para crear una herencia de clases hay que utilizar la palabra reservada **extends**.

Para darle el valor recibido como argumento en el constructor del hijo a la propiedad heredada del padre se usa la función **super(parámetroPadre)**.

```
class Vehiculo {  
    constructor(marca, modelo) {  
        this.marca = marca;  
        this.modelo = modelo;  
    }  
}  
class Coche extends Vehiculo{  
    constructor(marca, modelo, traccion) {  
        super(marca, modelo);  
        this.traccion = traccion;  
    }  
}
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.4. OBJETOS

Herencia - Métodos

Una clase creada con herencia hereda todos los métodos de su superclase.

```
class Vehiculo {
    constructor(marca, modelo) {
        this.marca = marca;
    }
    alertaEntradaVehiculo() {
        return "Ha llegado un nuevo vehículo " + this.marca;
    }
}
class Coche extends Vehiculo {
    constructor(marca, traccion) {
        super(marca);
        this.traccion = traccion;
    }
    alertaEntradaCoche() {
        return this.alertaEntradaVehiculo() + " con tracción " + this.traccion;
    }
}
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.4. OBJETOS

Métodos estáticos

Utilizamos **static** para definir un método estático en una clase.

Un método estático se llama directamente sin instanciar la clase.

Se suelen utilizar para crear funciones útiles en una aplicación.

No es necesario crear un objeto para llamar a un método estático.

```
class Rectangulo{
  constructor(x, y) {
    this._x = x;
    this._y = y;
  }
  static area (a, b) {
    return a * b;
  }
}
let rectangulo1 = new Rectangulo (2, 3);
console.log(rectangulo1.area(2, 3); //Daría error

console.log(Rectangulo.area(2, 3)); //Devolvería 6
```



4. PROGRAMACIÓN CON ARRAYS, FUNCIONES Y OBJETOS DEFINIDOS POR EL USUARIO

4.4. OBJETOS

Objetos literales:

Un literal es un valor fijo. Está formado por parejas de tipo nombre:valor.

Ejemplo:

```
coche {marca:"Ibiza", modelo:"Seat", combustible:"diesel", cantidad:40};
```

Ejemplo equivalente:

```
var coche = new Object();  
coche.marca="Ibiza";  
coche.modelo="Seat";  
coche.combustible="diesel";  
coche.cantidad=40;
```

Acceso:

```
coche.marca;  
coche["marca"]  
;
```

